Italian Lemmatisation with Neural Networks

Laurea Magistrale in Informatica

Report Natural Language Processing

A.A. 2022/2023

Filippo Bartolucci
filippo.bartolucci2@studio.unibo.it

Alfonso Esposito
alfonso.esposito5@studio.unibo.it

Contents

1 Introduction **5**
1.1 Lemmatisation 5
1.2 Italian language 6

2 Data and processing **7**
2.1 Data processing 7
2.2 Context encoding 8
2.3 Tag encoding 8
2.4 Word encoding 9
2.5 Lemma encoding 9

3 Model **11**
3.1 Model Architecture 11
3.2 Custom Accuracy 13
3.3 Training and validation 13

4 Result and discussion **15**
4.1 Partecipants 15
4.2 Open Class Words 15
4.3 Evaluation 16
4.4 Conclusion 17

1. Introduction

The following project was completed as part of the exam for the **Natural Language Processing** course at the University of Bologna. The aim of the project was to tackle the challenge of lemmatisation for the Italian language. We followed the guidelines of the **EVALITA 2011** task and conducted experiments using the provided dataset. In this project, we will describe the machine learning algorithm we used, including a brief explanation of the neural network, and present the results of our experiments.

1.1 Lemmatisation

Lemmatisation is a process in Natural Language Processing (NLP) where words are reduced to their base form, or lemma. The lemma is the root form of a word, which can be used to represent multiple inflected forms. This process is important in NLP as it helps in reducing the dimensionality of the text data and helps in standardizing the words to a common form.

$$\text{correndo} \rightarrow \text{correre}$$
$$\text{amici} \rightarrow \text{amico}$$

However, lemmatisation can be a challenging task, as it requires a deep understanding of the language grammar, context, and the relationship between words. One of the difficulties in lemmatisation is caused by ambiguous words that have multiple meanings or senses and can have different lemmas depending on the context in which they are used. Hence, lemmatisation is a crucial but challenging problem in NLP.

Lemmatisation and Parts of Speech (PoS) tagging are closely related in NLP as PoS tagging involves labeling words with their grammatical category. PoS information

is often used to inform the lemmatisation process. As a result, lemmatisation is often considered a sub-product of PoS tagging.

1.2 Italian language

The process of lemmatization in Italian can prove to be challenging due to the language's intricate inflectional system and the potential for ambiguity in sentences. The richness of the inflectional system means that the same word can have multiple forms depending on context, making it difficult to identify the correct lemma. The possibility of ambiguity, where a single word can have different lemmas based on the context in which it appears, further complicates the task, making lemmatization in Italian a complex challenge to overcome

Word Form	**PoS-tag**	**Possible Lemmas**
cannone	NOUN	cannone, canna
morti	NOUN	morto, morte
regione	NOUN	regione, regia
aria	NOUN	aria, ario
macchina	NOUN	macchina, macchia
piccione	NOUN	piccione, piccia
matematica	NOUN	matematica, matematico
passano	VERB	passare, passire
danno	VERB	dannare, dare

Table 1.1: Example of ambigous words

2. Data and processing

EVALITA 2011 offers two datasets for solving the lemmatization task: the Development Set (DS) and the Test Set (TS)[1]. The DS consists of 17313 tokens from 703 sentences, while the TS contains 133756 tokens from 5596 sentences. Each token in both datasets is accompanied by its PoS tag and the corresponding lemma. The DS is utilized to train the neural network, and the TS is employed to evaluate the performance of the task.

2.1 Data processing

The lemma of a word is dependent on both its morphology and the context in which it is used. Thus, we chose to use a combination of the word form, its PoS tag, and the context surrounding it as input to our neural network model.
This approach takes into consideration both the morphological features and contextual information, ensuring a more accurate lemmatization result. By incorporating all of these factors, our model should be able to disambiguate words and provide a more accurate lemma prediction.

Word Form	**PoS-tag**	**Lemmas**	**Context**
albanese	ADJ	albanese	investimento spericolati faticavano a pagare gli interessi , il lek , la moneta **albanese**, aveva toccato i massimi sul dollaro .
agguati	NOUN	agguato	certo fatto con quell' invocazione : non è nel suo carattere temere gli **agguati**.
dolore	NOUN	dolore	loro soffrono il **dolore** del sentimento che dura sempre , notte e giorno ;

Table 2.1: Example of word context

2.2 Context encoding

To accurately determine the lemma of a word, it is necessary to understand its semantic meaning within a specific context. To account for this, our model considers the contextual information surrounding the target word. This context is defined as a fixed number of words on both sides of the target word, and is defined by special tokens indicating the beginning and end of the contextual window.

<PRE>	io	ci	devo	**mettere**	la	massima	fiducia	<POST>

Figure 2.1: Context

We encode each unique word as an integer for input into our neural network. This is accomplished by using word2vec to obtain the weights for our embedding layer, converting each word into a numerical representation that can be processed by the network.
Word2Vec is a model that learns dense representations of words in a high-dimensional space, so that words with similar meanings are close in the vector space. By initializing the weights of an embedding layer in our target model with word2vec, our model can take advantage of the pre-trained knowledge of word relationships learned by word2vec.

2.3 Tag encoding

In addition to the words, our model also takes into account a sequence of PoS tags for each word in the context window. To simplify the input for our model, we encoded each of the 31 unique PoS tags as a unique integer, treating each tag as a separate class in the model for processing and analysis.

Context	soggetto	a	una	regola	inesorabile
Tags	adj	prep	art	nn	adj

Figure 2.2: Context and tags sequence

2.4 Word encoding

In order to better understand the morphological structure of the target word, we chose to encode it character-by-character into integer values. This encoding method transforms the word into an array of integers, which can be processed and analyzed within our model. To further enhance the representation of the characters, we have incorporated an embedding layer within the model. This layer allows the model to learn the most important weights for each character, which results in more accurate predictions. By encoding the target word in this manner, we are able to take into account its morphological structure.

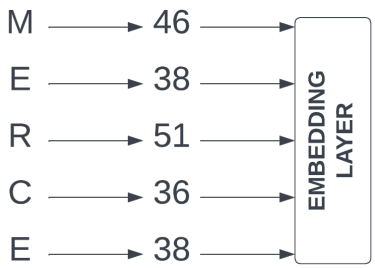

Figure 2.3: Example of word embedding at char level

2.5 Lemma encoding

In our quest to predict a lemma from context, word, and tag, we initially treated it as a multi-class classification problem. In this approach, each lemma was treated as a separate class. However, after conducting a series of experiments, it became clear that this method was not feasible. The main reason being that the number of classes was extremely high, which made it difficult to train the model with a limited number of samples. Furthermore, the test results showed that this method did not yield a desirable accuracy.

To overcome this challenge, we decided to change our approach and adopt a new strategy. We encoded each lemma character by character, using one-hot encoding. This not only reduced the dimensionality of the output, but it also allowed us to treat the problem as a sequence-to-sequence task. The model was able to learn the relationships between the input and output in a more meaningful way. This change in approach significantly improved the results, allowing the model to better predict the lemma from the context, word, and tag.

3. Model

The proposed model architecture is a deep neural network that combines the context of a word, its corresponding tags, and the word itself as inputs. The model consists of three different branches, each of which processes a different input.

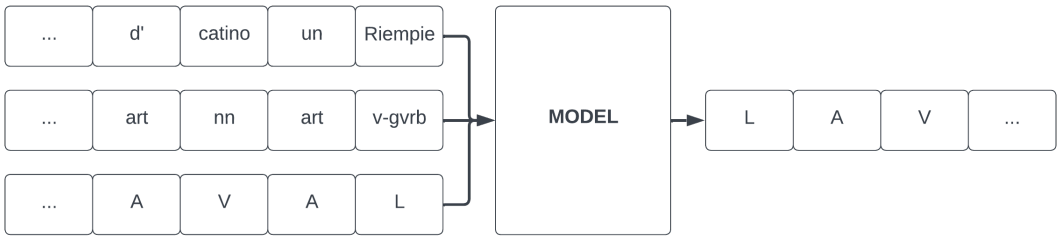

Figure 3.1: Inputs

3.1 Model Architecture

The first branch is for the context input which is passed through an embedding layer, which maps each word in the context to a high-dimensional vector representation. The embedding layer is initialized with pre-trained weights, and its training is set to be frozen. After the embedding, the context input is passed through a bidirectional LSTM layer, which encodes the context information into a compact representation.

The second branch is for the tag input. A second embedding layer is applied to the tag input, which maps each tag to a low-dimensional vector representation. The tag input is then passed through a bidirectional LSTM layer and the output is then passed through a TimeDistributed dense layer.

The third branch is for the word input. Then, the word input is passed through a third embedding layer, which maps each word to a high-dimensional vector

representation. This embedding layer is trained during the model's training. The word input is then passed through a bidirectional LSTM layer.

The outputs of the three branches are then combined using an attention mechanism, which computes the weighted sum of the outputs of the context and tag branches. The attention mechanism helps the model to focus on the most important information. The result of the attention mechanism is concatenated with the word input, and the combined representation is passed through two consecutive bidirectional LSTM layers. This allows the model to capture complex relationships between the inputs.

Finally, the output of the second LSTM layer is passed through multiple dense layers, each with a dropout layer to prevent overfitting. The dropout layer randomly drops out a portion of the neurons during training, which helps to reduce overfitting. The final output of the model is a sequence of softmax activations, which represents the predicted lemma for each word in the input.

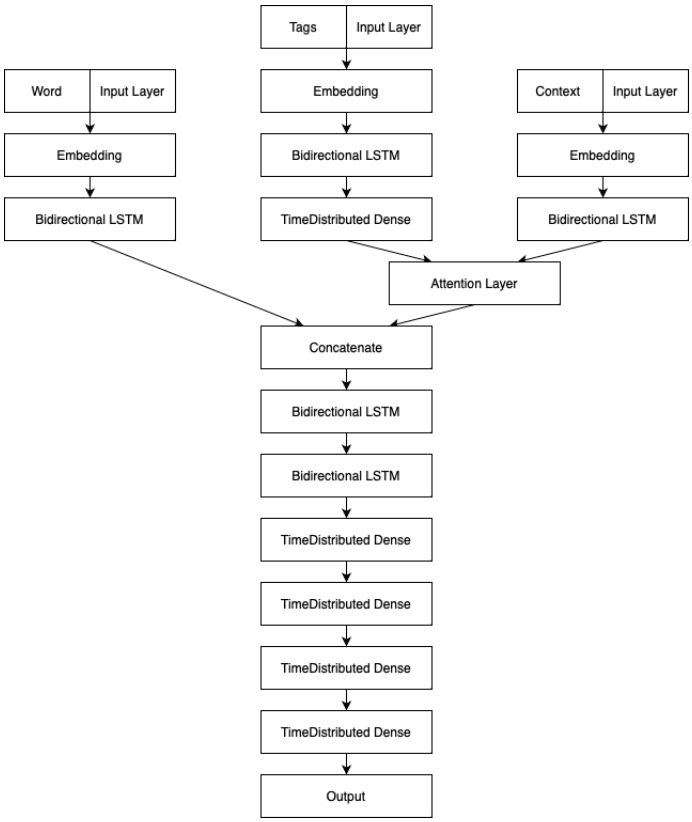

Figure 3.2: Our model

3.2 Custom Accuracy

To accurately evaluate the performance of our model in predicting lemmas character by character, we have created a custom accuracy function.

```
def accuracy(y_true, y_pred):
    y_true = tf.argmax(y_true, axis=-1)
    y_pred = tf.argmax(y_pred, axis=-1)
    correct_predictions = tf.reduce_all(tf.equal(y_true, y_pred), axis=-1)
    accuracy = tf.reduce_mean(tf.cast(correct_predictions, tf.float32))
    return accuracy
```

This function only considers a lemma prediction to be correct if every character in the generated sequence matches the expected result, rather than just evaluating based on the number of correctly predicted characters. The aim is to measure the accuracy of the lemma prediction as a whole, not just individual characters.

3.3 Training and validation

The dev set is composed of 8,466 open class words, each accompanied by its context and associated PoS tags. Due to the significant size of the dev set, we made the decision to allocate only a small portion of it as a validation set. In particular, we set aside just 0.05 of the dev set to serve as the validation set, as a larger validation set would have required a larger amount of data for the training of our network. Despite its smaller size, the validation set provides an important measure of the model's accuracy and helps us fine-tune our model for improved performance.

The validation set and early stopping technique were used to identify the model with the best generalization capacity. By monitoring the performance of the model on the validation set during training, we were able to determine when the model had stopped improving and was at risk of overfitting to the training data. By using early stopping, we were able to prevent overfitting and select the model with the best generalization performance, ensuring that it would perform well on unseen data.

4. Result and discussion

The performance of our model will be evaluated using a test set shared among the participants of the EVALITA 2011 task, and will be compared against the results of other models.

4.1 Partecipants

These are the four participants in the EVALITA 2011 competition.

Name	**Institution**	**System Label**
Rodolfo Delmonte	University of Venice, Italy	Delmonte_UniVe
Djamè Seddah	Alpage (Inria)/Univ. Paris Sorbonne, France	Seddah_Inria-UniSorbonne
Maria Simi	University of Pisa, Italy	Simi_UniPi
Fabio Tamburini	University of Bologna, Italy	Tamburini_UniBo

Table 4.1: Partecipants

4.2 Open Class Words

In our evaluation of the model, we are focused solely on the accuracy of predicting the lemma of open-class words. Open-class words represent the most complex and interesting cases in natural language processing as they are constantly evolving and expanding, unlike closed-class or functional words that have a more limited

number of words and do not change as often. Thus, evaluating the performance of the model in predicting the lemma of open-class words provides a more thorough understanding of its ability to disambiguate words in real-world text.

4.3 Evaluation

The tables below presents a comparison of accuracy, absolute error and relative error of our model with other participants in the Evalita2011 evaluation.

Table 4.2: Results

System	Lemmatisation Accuracy
Simi_UniPi	99.06%
Tamburini_UniBo	98.74%
Delmonte_UniVe	98.42%
Our Model	**95.07%**
Seddah_Indria-UniSorbonne	94.76%

Table 4.3: Systems' absolute error distribution with respect to PoS-tags (computed as the error for each class divided by the total number of errors made by the system

System	**ADJ_***	**ADV**	**NN**	**V_***
Simi_UniPi	15.6%	8.2%	61.2%	15.0%
Tamburini_UniBo	17.7%	5.1%	64.4%	12.8%
Delmonte_UniVe	11.9%	6.7%	70.8%	10.6%
Our Model	**12.8%**	**4.5%**	**29.7%**	**53.1%**
Seddah_Indria-UniSorbonne	25.6%	4.9%	30.4%	44.1%

Table 4.4: Systems' relative error inside each lexical class (computed as the error made by the system for each class divided by the total number of token in the same class contained into the TS

System	**ADJ_***	**ADV**	**NN**	**V_***
Simi_UniPi	0.8%	0.7%	1.4%	0.5%
Tamburini_UniBo	1.2%	0.6%	2.0%	0.5%
Delmonte_UniVe	1.0%	1.0%	2.7%	0.6%
Our Model	**3.1%**	**2.0%**	**3.4%**	**8.2%**
Seddah_Indria-UniSorbonne	7.0%	2.4%	3.9%	8.1%

Compared to the other participants in EVALITA 2011, our system has demonstrated better performance in certain classifications, but has struggled with correctly lemmatizing verbs. This could be due to the limitations in the dataset used and the challenging nature of the Italian language, which has a multitude of complex verbal forms.

4.4 Conclusion

In conclusion, our model demonstrated a good level of accuracy when compared to the other participants in the Evalita2011 competition. While our model slightly outperformed the solution developed from Inria-UniSorbonne, which is based on a neural network, it still fell short in comparison to other participants who utilized more traditional solutions. It's important to note that one limitation of our model is the limited size of the development set, which may have impacted the final results. Nevertheless, this experiment highlights the potential for further improvement and optimization of our model in future iterations.

Bibliography

[1] Fabio Tamburini. "The Lemmatisation Task at the EVALITA 2011 Evaluation Campaign". In: *Evaluation of Natural Language and Speech Tools for Italian*. Ed. by Bernardo Magnini et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 230–238. ISBN: 978-3-642-35828-9.